

Entrega 2 Proyecto final

Análisis y diseño de algoritmos

Índice

Solución Greedy	2
Idea general del algoritmo.....	2
Decisión local óptima.....	2
Análisis de complejidad — Greedy.....	3
Análisis crítico — Greedy	5
Conclusión — Greedy	6
Solución con Backtracking	7
Idea general del algoritmo.....	7
Poda implementada	7
Restricciones utilizadas para podar	8
Análisis de complejidad — Backtracking.....	8
Medición empírica de tiempos — Backtracking	9
Análisis	9
Conclusión — Backtracking.....	9
Comparativa parcial	10

Entrega 2 – Greedy y Backtracking

Solución Greedy

Idea general del algoritmo

La estrategia greedy utilizada en este proyecto busca resolver el problema de asignación de paquetes tomando decisiones rápidas y locales en cada paso del proceso.

El algoritmo recorre los paquetes en orden y, para cada uno, intenta asignarlo inmediatamente al primer camión disponible donde el peso no exceda la capacidad restante.

La idea principal es:

Maximizar rápidamente la cantidad de paquetes entregados sin explorar múltiples combinaciones posibles.

A diferencia de fuerza bruta o backtracking, greedy no analiza escenarios futuros ni reconsidera decisiones tomadas anteriormente. Una vez un paquete es asignado, esa decisión permanece fija durante toda la ejecución.

Decisión local óptima

La decisión local óptima implementada consiste en:

“Asignar el paquete actual al primer camión donde pueda ser transportado sin superar la capacidad máxima.”

Esto permite reducir drásticamente el tiempo de ejecución, ya que evita explorar todas las configuraciones posibles del problema.

Sin embargo, esta decisión se toma únicamente con la información disponible en el momento actual, sin evaluar cómo puede afectar futuras asignaciones.

Funcionamiento del algoritmo

El procedimiento general del algoritmo greedy es el siguiente:

1. Se inicializan las cargas actuales de todos los camiones.
2. Se recorren los paquetes uno por uno.
3. Para cada paquete:
 - Se revisan los camiones en orden.
 - Si el paquete cabe en un camión:
 - Se asigna inmediatamente.
 - Se actualiza la carga ocupada.
 - Si no cabe en ningún camión:
 - El paquete se descarta.
4. El proceso continúa hasta procesar todos los paquetes.

El algoritmo nunca retrocede ni reorganiza paquetes ya asignados.

Análisis de complejidad – Greedy

Complejidad temporal

Sea:

- n = cantidad de paquetes
- m = cantidad de camiones

El algoritmo primero ordena los paquetes por prioridad, lo que tiene un costo de $O(n \log n)$. Luego recorre los paquetes uno por uno y para cada uno revisa hasta m camiones.

Por lo tanto, la complejidad temporal es:

$O(n \log n + n * m)$ que se simplifica a **$O(n * m)$** para valores grandes de m .

Justificación paso a paso

1. Se ordenan los n paquetes por prioridad $O(n \log n)$
2. El algoritmo recorre los n paquetes
3. Para cada paquete puede revisar hasta m camiones
4. Cada verificación de capacidad es una operación constante
5. Total: $O(n \log n + n * m)$ - **$O(n * m)$**

Complejidad espacial

Se almacena un arreglo de cargas para los m camiones y la asignación de hasta n paquetes.

$O(n + m)$

Medición empírica de tiempos — Greedy

Caso	Paquetes	Camiones	Combinaciones	Tiempo real
Pequeño	80	15	$80 * 15 = 1.200$	0.000128 seg
Mediano	700	120	$120 * 700 = 84.700$	0.001667 seg
Grande	5000	900	$900 * 5000 = 4.500.000$	No termina

Análisis crítico – Greedy

La principal ventaja del enfoque greedy es su velocidad. El algoritmo toma decisiones inmediatas basadas en la prioridad de cada paquete, evitando explorar combinaciones complejas.

Sin embargo, esta rapidez tiene un costo importante: greedy no garantiza encontrar la solución óptima. El problema ocurre porque el algoritmo solo analiza el estado actual y no considera cómo una decisión puede afectar futuras asignaciones.

Contraejemplo

Supongamos:

Camiones

Camion	Capacidad
C1	10
C2	10

Paquetes

Paquete	Peso	Prioridad
P1	9	3
P2	9	2
P3	2	1
P4	2	1

Resultado usando Greedy

Greedy ordena por prioridad y asigna

P1 - C1,

P2 - C2.

Ambos camiones quedan con capacidad restante de 1,

P3 y P4 no caben.

Resultado: 2 paquetes entregados.

Solución óptima

Existe una distribución mejor:

- $P1 + P3 \rightarrow C1$
- $P2 + P4 \rightarrow C2$

Resultado final:

- 4 paquetes entregados.

Esto demuestra que la estrategia greedy puede desperdiciar capacidad útil y perder soluciones mejores.

Conclusión — Greedy

El algoritmo greedy representa una solución eficiente para el problema de asignación de paquetes.

Su decisión local óptima — asignar primero los paquetes de mayor prioridad — mejora la calidad de la solución respecto a una asignación sin criterio, pero no garantiza optimalidad.

Por esta razón, greedy es adecuado cuando:

- Se prioriza rapidez
- El tiempo de respuesta es crítico
- El tamaño de entrada hace inviables soluciones exhaustivas.

Solución con Backtracking

Idea general del algoritmo

La estrategia de backtracking busca explorar múltiples combinaciones posibles de asignación de paquetes a camiones.

A diferencia de greedy, este enfoque sí reconsidera decisiones y evalúa distintas configuraciones antes de elegir la mejor solución.

El objetivo principal es:

Maximizar la cantidad total de paquetes entregados respetando las restricciones de capacidad.

El algoritmo explora recursivamente diferentes posibilidades y conserva únicamente la mejor solución encontrada.

Funcionamiento del algoritmo

Para cada paquete existen varias decisiones posibles:

1. Asignarlo a un camión válido.
2. Probar otro camión.
3. O no asignarlo.

El algoritmo realiza:

- una decisión,
- explora recursivamente,
- y luego deshace la decisión para probar otras alternativas.

Este mecanismo permite recorrer el espacio de soluciones posibles de forma sistemática.

Poda implementada

Debido al enorme número de combinaciones posibles, el algoritmo incorpora técnicas de poda para reducir el espacio de búsqueda.

La poda principal utilizada consiste en verificar:

“¿Aún es posible superar la mejor solución encontrada?” Para

ello se calcula:

- la cantidad actual de paquetes asignados,
- y los paquetes restantes por procesar.

Si incluso asignando todos los paquetes restantes no es posible superar la mejor solución actual, la rama se elimina inmediatamente.

Restricciones utilizadas para podar

1. Restricción de capacidad

Un paquete solo puede asignarse si no supera la capacidad restante del camión. Esto evita generar soluciones inválidas.

2. Poda por cota superior

Si: *paquetes actuales* + *paquetes restantes* no supera la mejor solución encontrada la rama deja de explorarse. Esta poda reduce considerablemente la cantidad de combinaciones evaluadas.

Análisis de complejidad – Backtracking

Complejidad temporal (peor caso)

Sea:

- n = cantidad de paquetes
- m = cantidad de camiones

Cada paquete puede asignarse a cualquiera de los m camiones, o no asignarse. Por tanto, el número aproximado de combinaciones posibles es: $(m + 1)^n$. La complejidad temporal en el peor caso es: $O((m + 1)^n)$, Esto corresponde a crecimiento exponencial.

Complejidad espacial

El algoritmo almacena:

- Cargas actuales
- Asignaciones parciales
- Llamadas recursivas

La complejidad espacial aproximada es:

$$O(n * m)$$

Caso promedio con poda

Aunque teóricamente el algoritmo sigue siendo exponencial, en la práctica las podas reducen significativamente el número de ramas exploradas.

Esto produce mejoras importantes respecto a fuerza bruta.

Sin poda, el algoritmo sería prácticamente inviable para entradas medianas.

Medición empírica de tiempos – Backtracking

Caso	Paquetes	Camiones	Tiempo real
Pequeño	80	15	0.001530 seg
Mediano	700	120	0.3733164 seg
Grande	5000	900	No termina

Análisis

Backtracking obtiene mejores soluciones que greedy debido a que explora múltiples configuraciones posibles.

Sin embargo, esto incrementa considerablemente el costo computacional.

A medida que crece la cantidad de paquetes y camiones, el número de combinaciones aumenta exponencialmente.

Las podas permiten reducir parte de esta explosión combinatoria, pero no eliminan completamente el problema.

Conclusión – Backtracking

Backtracking ofrece una solución mucho más precisa que greedy y cercana a la optimalidad.

La incorporación de podas mejora significativamente el rendimiento práctico del algoritmo.

No obstante, debido al crecimiento exponencial del espacio de búsqueda, el algoritmo sigue siendo costoso para entradas grandes.

Por esta razón, backtracking resulta adecuado cuando:

- Se busca maximizar calidad de solución,
- El tamaño de entrada es moderado,
- Y se dispone de mayor tiempo de procesamiento.

Comparativa parcial

Algoritmo	Complejidad	Optimo?	Velocidad	Escalabilidad
Fuerza Bruta	Exponencial	Sí	Muy lenta	Muy baja
Recursivo	$O(n * m)$	No	Rápida	Media
Greedy	$O(n * m)$	No	Muy rápida	Alta
Backtracking	Exponencial	Sí	Media/Lenta	Baja

Discusión general

La diferencia principal entre los algoritmos radica en el equilibrio entre: velocidad, calidad de solución, y costo computacional.

Greedy prioriza rapidez sacrificando optimalidad.

Backtracking prioriza calidad de solución sacrificando rendimiento.

Fuerza bruta garantiza optimalidad, pero resulta inviable para entradas reales.

Recursivo es rápido y simple pero no garantiza optimalidad y falla en entradas grandes por el límite de llamadas recursivas.

Por ello, cada estrategia representa un trade-off diferente dependiendo de las necesidades del problema.